

信号 1（进程同步）

1、信号由操作系统向进程发送SIGINT，当进程收到信号：查看信号编号：kill -l

```
xingwenpeng@ubuntu:~/LinuxCode/ProIPC-04$ kill -l
 1) SIGHUP          2) SIGINT          3) SIGQUIT         4) SIGILL          5) SIGTRAP
 6) SIGABRT         7) SIGBUS         8) SIGFPE          9) SIGKILL         10) SIGUSR1
11) SIGSEGV        12) SIGUSR2        13) SIGPIPE        14) SIGALRM        15) SIGTERM
16) SIGSTKFLT      17) SIGCHLD       18) SIGCONT        19) SIGSTOP        20) SIGTSTP
21) SIGTTIN        22) SIGTTOU       23) SIGURG         24) SIGXCPU        25) SIGXFSZ
26) SIGVTALRM      27) SIGPROF       28) SIGWINCH       29) SIGIO          30) SIGPWR
31) SIGSYS         34) SIGRTMIN      35) SIGRTMIN+1    36) SIGRTMIN+2    37) SIGRTMIN+3
38) SIGRTMIN+4    39) SIGRTMIN+5    40) SIGRTMIN+6    41) SIGRTMIN+7    42) SIGRTMIN+8
43) SIGRTMIN+9    44) SIGRTMIN+10   45) SIGRTMIN+11   46) SIGRTMIN+12   47) SIGRTMIN+13
48) SIGRTMIN+14   49) SIGRTMIN+15   50) SIGRTMAX-14   51) SIGRTMAX-13   52) SIGRTMAX-12
53) SIGRTMAX-11   54) SIGRTMAX-10   55) SIGRTMAX-9    56) SIGRTMAX-8    57) SIGRTMAX-7
58) SIGRTMAX-6    59) SIGRTMAX-5    60) SIGRTMAX-4    61) SIGRTMAX-3    62) SIGRTMAX-2
63) SIGRTMAX-1    64) SIGRTMAX
```

采用信号实现进程间的同步。讨论前31个信号。

按下 Ctrl+C，发送了SIGINT信号，表示终止当前进程。

信号有三种处理方式：

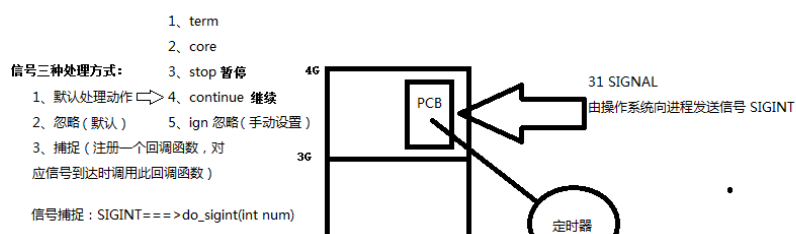
1、默认处理动作（默认使用）：

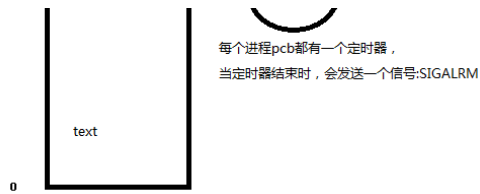
- 1、term：终止当前的进程
- 2、core：当前进程结束后，生成core文件。死后验尸
- 3、stop：暂停当前进程（ctrl + z -> SIGTSTP）
- 4、continue：继续执行当前进程（fg -> SIGCONT）
- 5、ign：当前进程忽略这个信号（手动操作）

2、忽略：手动操作有没有开启

3、捕捉：注册一个回调函数，对应信号到达时调用此回调函数。

查看信号机制：man 7 signal





2、信号产生：

- a) SIGSTOP 19号 系统暂停
- b) SIGFPE 8号 除零操作
- c) SIGQUIT 3号 ctrl + \ 退出进程

信号产生有：

- 1、终端特殊按键
- 2、硬件异常：除 0 操作，访问非法内存（段错误 SIGSEGV 11号）
- 3、kill函数或kill命令

管道读端关闭，写端写数据，会产生一个信号SIGPIPE

SIGUSE1\SIGUSE2用户自定义的信号。

SIGCHLD 子进程结束时，父进程会收到这个信号。默认为忽略这个信号

3、kill函数sig = 0 是用于检测进程是否存在，但存在不安全性。可能下一秒被杀了。

```
1 int kill(pid_t pid, int sig);
2
3 a)pid > 0 时，sig发送给PID为pid的进程；
4
5 b)pid == 0 时，sig发送给与发送进程同组的所有进程；
6
7 c)pid < 0 时，sig发送给组ID为|-pid|的进程，并且发送进程具有向其发送信号的权限；
8
9 d)pid == -1 时，sig发送给进程有权限向他们发送信号系统上的所有进程。比如root可以发送所有的用
10
```

```
11 sig = 0时, 是用于检测进程是否存在, 不存在返回 -1, 但存在不确定性。
```

4、raise函数当前进程发送信号

```
1 int raise(int sig);
```

5、abort函数给自己进程发送信号 SIGABRT 6号

```
1 void abort(void);
```

6、alarm函数定时器

每个进程只有一个定时器SIGALRM 14号, 定时结束进程也结束。

```
1 unsigned int alarm(unsigned int seconds);
```

当定时结束时发送一个信号, 随之响应进程结束, 结束时会返回剩余时间。

```
1 #include <stdio.h>
2 #include <unistd.h>
3
4 int main(void)
5 {
6     int count = 0;
7     alarm(1); // 设定定时器
8     while(1)
9     {
10         printf("%d ", ++count);
```

```
11     }
12     return 0;
13 }
```

6.5、信号

- 1 1) **SIGHUP**: 当用户退出shell时, 由该shell启动的所有进程将收到这个信号, 默认动作为终止进程
- 2
- 3 2) **SIGINT**: 当用户按下了<Ctrl+C>组合键时, 用户终端向正在运行中的由该终端启动的程序发出此信号。
- 4
- 5 3) **SIGQUIT**: 当用户按下<ctrl+\>组合键时产生该信号, 用户终端向正在运行中的由该终端启动的程序发
- 6
- 7 4) **SIGILL**: CPU检测到某进程执行了非法指令。默认动作为终止进程并产生core文件
- 8
- 9 5) **SIGTRAP**: 该信号由断点指令或其他trap指令产生。默认动作为终止进程并产生core文件。
- 10
- 11 6) **SIGABRT**: 调用abort函数时产生该信号。默认动作为终止进程并产生core文件。
- 12
- 13 7) **SIGBUS**: 非法访问内存地址, 包括内存对齐出错, 默认动作为终止进程并产生core文件。
- 14
- 15 8) **SIGFPE**: 在发生致命的运算错误时发出。不仅包括浮点运算错误, 还包括溢出及除数为0等所有的算法
- 16
- 17 9) **SIGKILL**: 无条件终止进程。本信号不能被忽略, 处理和阻塞。默认动作为终止进程。它向系统管理员:
- 18
- 19 10) **SIGUSE1**: 用户定义的信号。即程序员可以在程序中定义并使用该信号。默认动作为终止进程。
- 20
- 21 11) **SIGSEGV**: 指示进程进行了无效内存访问。默认动作为终止进程并产生core文件。
- 22
- 23 12) **SIGUSR2**: 这是另外一个用户自定义信号, 程序员可以在程序中定义并使用该信号。默认动作为终止进
- 24
- 25 13) **SIGPIPE**: Broken pipe向一个没有读端的管道写数据。默认动作为终止进程。
- 26
- 27 14) **SIGALRM**: 定时器超时, 超时的时间由系统调用alarm设置。默认动作为终止进程。
- 28
- 29 15) **SIGTERM**: 程序结束信号, 与SIGKILL不同的是, 该信号可以被阻塞和终止。通常用来要求程序正常退
- 30
- 31 16) **SIGCHLD**: 子进程结束时, 父进程会收到这个信号。默认动作为忽略这个信号。
- 32
- 33 17) **SIGCONT**: 停止进程的执行。信号不能被忽略, 处理和阻塞。默认动作为终止进程。
- 34

35 18) SIGTTIN: 后台进程读终端控制台。默认动作为暂停进程。
36
37 19) SIGTSTP: 停止进程的运行。按下<ctrl+z>组合键时发出这个信号。默认动作为暂停进程。
38
39 21) SIGTTOU: 该信号类似于SIGTTIN, 在后台进程要向终端输出数据时发生。默认动作为暂停进程。
40
41 22) SIGURG: 套接字上有紧急数据时, 向当前正在运行的进程发出些信号, 报告有紧急数据到达。如网络。
42
43 23) SIGXFSZ: 进程执行时间超过了分配给该进程的CPU时间, 系统产生该信号并发送给该进程。默认动作
44
45 24) SIGXFSZ: 超过文件的最大长度设置。默认动作为终止进程。
46
47 25) SIGVTALRM: 虚拟时钟超时时产生该信号。类似于SIGALRM, 但是该信号只计算该进程占用CPU的使用
48
49 26) SIGIPROF: 类似于SIGVTALRM, 它不公包括该进程占用CPU时间还包括执行系统调用时间。默认动作为
50
51 27) SIGWINCH: 窗口变化大小时发出。默认动作为忽略该信号。
52
53 28) SIGIO: 此信号向进程指示发出了一个异步IO事件。默认动作为忽略。
54
55 29) SIGPWR: 关机。默认动作为终止进程。
56
57 30) SIGSYS: 无效的系统调用。默认动作为终止进程并产生core文件。
58
59 31) SIGRTMIN~ (64) SIGRTMAX: LINUX的实时信号, 它们没有固定的含义(可以由用户自定义)。所有

7、信号集

a)未决信号集

- i. 只可读不可写, 状态位自动翻转
- ii. 1 表示有信号到达, 0 表示没有信号或已响应此信号

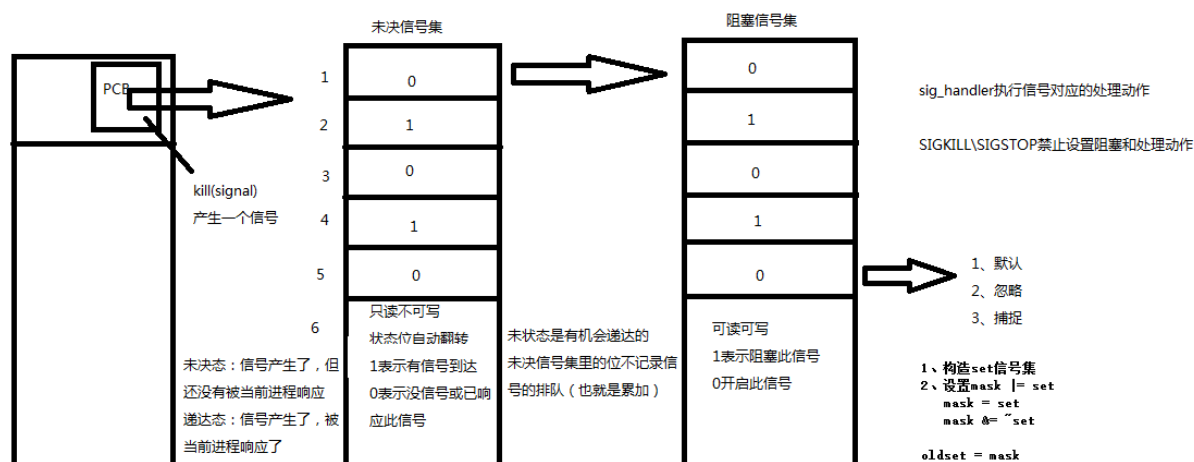
b)阻塞信号集

- 1) 可读可写
- 2) 1表示阻塞此信号, 0表示开启此信号

除了SIGKILL/SIGSTOP 禁止设置阻塞和处理动作。

信号屏蔽也称为阻塞信号。

内核发送一个信号，到达 未决信号集置 1，然后往上传达 到 阻塞信号集，如果该信号对应的阻塞信号 置1，那么该信号不能通过。如果是 0，则可以通过。然后处理，为决态信号置 0。



8、信号集处理函数

`sigset_t`为信号集，可用`sizeof(sigset_t)`查看大小

```
1 int sigemptyset(sigset_t *set); //置空sigset_t结构体
2
3 int sigfillset(sigset_t *set); //填充sigset_t结构体置1
4
5 int sigaddset(sigset_t *set, int signo); //在结构体中添加信号，置1
6
7 int sigdelset(sigset_t *set, int signo); //在结构体中删除信号，置0
8
9 int sigismember(const sigset_t *set, int signo); //打印结构体中信号的状态
```

9、sigprocmask函数信号集（读写（设置）阻塞态信号集）

```
1 int sigprocmask(int how,const sigset_t *set,sigset_t *oset);
```

调用函数**sigprocmask**可以读取或更改进程的信号屏蔽字(阻塞信号集)。

- 1、如果**oset**是非空指针，读出进程当前信号的信号屏蔽的参数，传出参数。
- 2、如果**set**是非空指针，会更改进程的**信号屏蔽字**。
- 3、参数**how**指示如何更改。

how参数的含义：假设当前的信号屏蔽字为**mask**

- 1) SIG_BLOCK **set**包含了我们希望添加到当前信号屏蔽字的信号，相当于 $\text{mask}=\text{mask}|\text{set}$
- 2) SIG_UNBLOCK **set**包含了我们希望从当前信号屏蔽字中解除阻塞的信号，相当于 $\text{mask}=\text{mask}\&\sim\text{set}$
- 3) SIG_SETMASK 设置当前信号屏蔽字为**set**所指向的值，相当于 $\text{mask}=\text{set}$

```
1 #include <stdio.h>
2 #include <signal.h>
3
4 void print_pend(sigset_t pset)
5 {
6     int i;
7     for(i = 1; i < 32; i++)
8     {
9         if(sigismember(&pset,i) == 1)
10         {
11             putchar('1');
12         }
13         else
14         {
15             putchar('0');
16         }
17     }
18     puts(" ");
19 }
20
21 int main(void)
22 {
23     sigset_t set,pset;
```

```

24
25     sigemptyset(&set); //初始化信号集
26     sigaddset(&set, SIGINT); //这步没有设置阻塞，只是构造一个信号集
27     sigaddset(&set, SIGQUIT);
28     sigaddset(&set, SIGKILL); //不能阻塞该信号
29     sigprocmask(SIG_BLOCK, &set, NULL); //把构造好set设置为当前进程的阻塞信号集
30
31     while(1)
32     {
33         sigpending(&pset); //获取当前进程未决信号集
34         print_pend(pset);
35         sleep(1);
36     }
37
38     return 0;
39 }

```

10、sigpending函数（读取未决态信号集）

```

1 int sigpending(sigset_t *set);

```

sigpending 读取当前进程的未决信号集，通过 **set** 参数传出。调用成功则返回 0，出错则返回 -1。

```

1 #include <stdio.h>
2 #include <signal.h>
3
4 void print_pend(sigset_t pset)
5 {
6     int i;
7     for(i = 1; i < 32; i++)
8     {
9         if(sigismember(&pset, i) == 1)
10         {
11             putchar('1');
12         }
13         else

```

```

14     {
15         putchar('0');
16     }
17 }
18 puts(" ");
19 }
20
21 int main(void)
22 {
23     int n = 10;
24     sigset_t set, pset;
25
26     sigemptyset(&set);
27     sigaddset(&set, SIGINT); // 这步没有设置阻塞，只是构造一个信号集
28     sigaddset(&set, SIGQUIT);
29     sigaddset(&set, SIGKILL); // 不能阻塞该信号
30     sigprocmask(SIG_BLOCK, &set, NULL); // 把构造好set设置为当前进程的阻塞信号集
31
32     while(1)
33     {
34         sigpending(&pset); // 获取当前进程未决信号集
35         print_pend(pset);
36         sleep(1);
37         if(n == 0)
38         {
39             sigemptyset(&set);
40             sigaddset(&set, SIGINT);
41             sigprocmask(SIG_UNBLOCK, &set, NULL); // 把构造好set设置为当前进程的阻塞信号集，n
42         }
43         n--;
44     }
45
46     return 0;
47 }

```

在阻塞信号集设置信号的抵达控制，未决态信号集只是表达产生信号的标记。

11、信号捕捉设定

```

1 int sigaction(int signum, const struct sigaction *act, struct sigaction *oldact);

```

参数:

signum:捕捉的信号

act : 捕捉动作

oldact: 原来的动作(传出参数)

struct sigaction结构体定义:

```
1 struct sigaction
2 {
3     void (*sa_handler)(int);
4     void (*sa_sigaction)(int,siginfo_t *,void *);
5     sigset_t sa_mask;//与当前的 mask|sa_mask,临时有效的信号集
6     int sa_flags;
7     void (*sa_restorer)(void);//基本不用
8 };
```

sa_mask | mask | 阻塞当前信号 是临时有效, 执行完回调函数时, 回到main函数, 还是会还原回mask的状态。

1、 sa_handler:早期的捕捉函数

2、 sa_sigaction:新添加的捕捉函数, 可以参数。和sa_handler互斥。两者通过sa_flags选择采用种捕捉函数

3、 sa_flags: SA_SIGINFO 或者 0

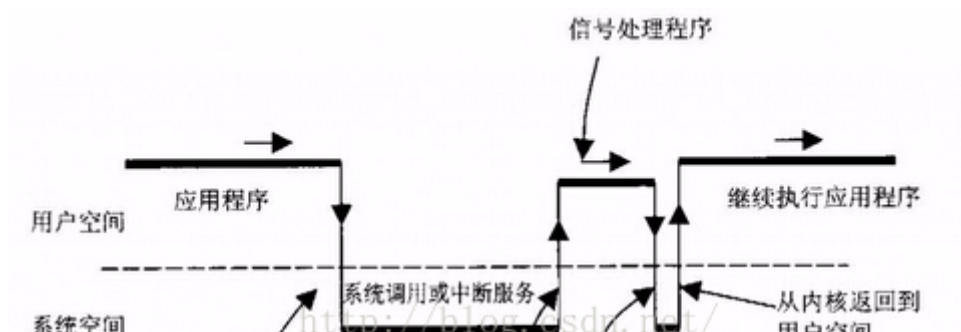
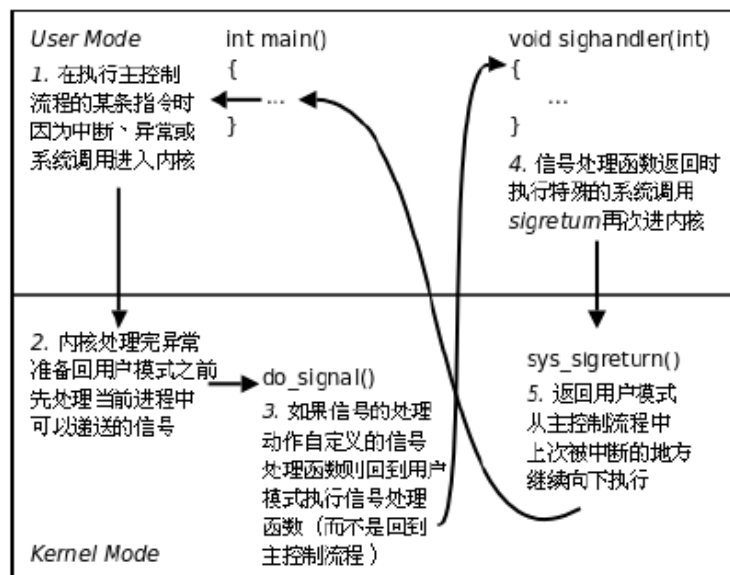
4、 sa_restorer: 保留, 已过时

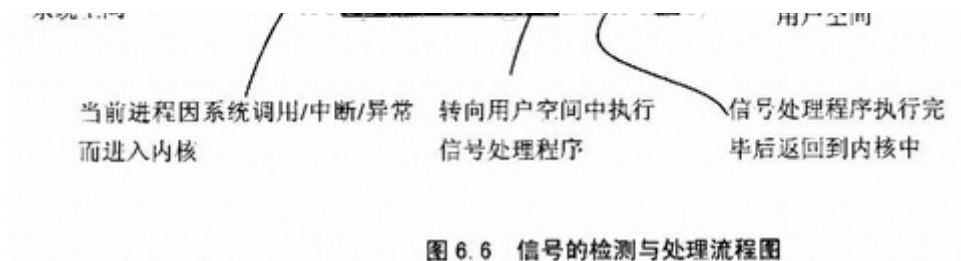
```
1 //sigaction.c
2 #include <stdio.h>
3 #include <unistd.h>
4 #include <signal.h>
5
6 void do_sigint(int num)
7 {
8     printf("%d  hello world\n",num);
```

```

9 }
10
11 int main(void)
12 {
13     struct sigaction act;
14     //struct sigaction *act;
15     //init struct act
16     act.sa_handler = do_sigint;
17     //act.sa_handler = SIG_IGN;//忽略捕捉
18     sigemptyset(&act.sa_mask);
19     act.sa_flags = 0;
20
21     sigaction(SIGINT,&act,NULL);
22     while(1)
23     {
24         printf("-----\n");
25         sleep(1);
26     }
27
28     return 0;
29 }

```





12、利用SIGUSR1和SIGUSR2实现父子进程同步输出

注意：子进程继承了父进程的信号屏蔽字和信号处理动作

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <signal.h>
4 #include <unistd.h>
5
6 int num;
7 pid_t pid;
8
9 void sys_err(char *str)
10 {
11     perror(str);
12     exit(1);
13 }
14
15 void do_sig_parent(int n)
16 {
17     printf("p %d\n", num);
18     num += 2;
19     sleep(1);
20     kill(pid, SIGUSR1); //子进程还没执行，就发送信号了，所有子进程就终止了。
21 }
22
23 void do_sig_child(int n)
24 {
25     printf("c %d\n", num);
26     num += 2;
27     kill(getppid(), SIGUSR1);
28 }
29

```

```

30 int main(void)
31 {
32     struct sigaction act;
33     //SIGUSR1
34     sigset_t set;
35     sigemptyset(&set);
36     sigaddset(&set,SIGUSR1);
37     sigprocmask(SIG_BLOCK,&set,NULL); //阻塞
38
39     if((pid = fork()) < 0)
40     {
41         sys_err("fork");
42     }
43     else if(pid == 0)
44     {
45         //child
46         act.sa_handler = do_sig_child;
47         sigemptyset(&act.sa_mask);
48         act.sa_flags = 0;
49
50         sigaction(SIGUSR1,&act,NULL);
51         num = 2;
52         sigprocmask(SIG_UNBLOCK,&set,NULL); //解除阻塞
53         while(1)
54         {
55             sleep(1);
56         }
57     }
58     else
59     {
60         //parent
61         act.sa_handler = do_sig_parent;
62         sigemptyset(&act.sa_mask);
63         act.sa_flags = 0;
64
65         sigaction(SIGUSR1,&act,NULL);
66         sigprocmask(SIG_UNBLOCK,&set,NULL); //解除阻塞
67         num = 1;
68         raise(SIGUSR1);
69         while(1)
70         {
71             sleep(1);
72         }
73     }

```

```
74
75     return 0;
76 }
```

13、C标准库信号处理函数

```
1 typedef void(*sighandler_t)(int);
2 sighandler_t signal(int signum,sighandler_t handler);
3
4 int system(const char *command);//集合fork, exec,wait一体
```

```
1 //signal.c
2 #include <stdio.h>
3 #include <signal.h>
4
5 void do_sig(int n)
6 {
7     printf("hello %d\n",n);
8 }
9
10 int main(void)
11 {
12     signal(SIGINT,do_sig);
13     while(1)
14     {
15         printf("-----\n");
16         sleep(1);
17     }
18     return 0;
19 }
```

```
1 //system.c
```

```

1 //system.c
2 #include <stdio.h>
3
4 int main(void)
5 {
6     system("ls -l");
7     return 0;
8 }

```

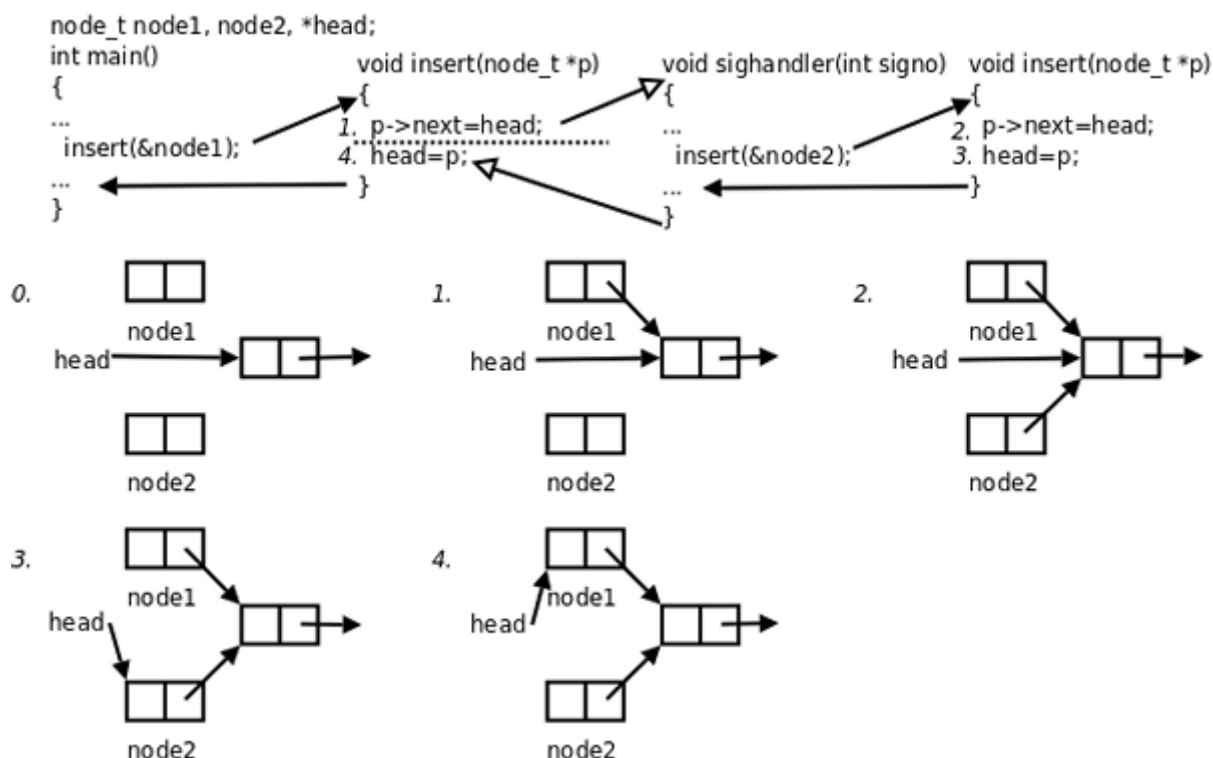
14、可重入函数

捕捉的函数避免使用不可重入函数，造成数据的混乱。

可重入函数是不包含全局变量和静态变量的函数。

在信号捕捉函数里应使用可重入函数。

Man 7 signal 查看系统可重入函数。**strtok**函数不可重入函数，**strtok_r**函数则可重入函数。



链表的头插法：在插入新节点时，进行了一半，然后来了个信号，打断了。然后信号的回调函数也进行新节点插入，那么就会出现错误。所以`insert`函数不可重入函数。

15、时序竞态

1) pause函数:

```
1 int pause(void);
```

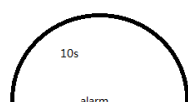
使调用进程挂起，直到有信号递达，如果递达信号是忽略，则继续挂起

```
1 #include <stdio.h>
2 #include <signal.h>
3
4 void do_sig(int n)
5 {
6
7 }
8
9 int main(void)
10 {
11     signal(SIGINT, do_sig);
12
13     printf("hello\n");
14     pause();
15     printf("world\n");
16 }
```

2) sigsuspend 函数

```
1 int sigsuspend(const sigset_t *mask); // (原子操作)
```

- 1、挂起等待
- 2、以通过指定mask(临时)来临时解除对某个信号的屏蔽
- 3、当被信号唤醒sigsuspend返回时，进程的信号屏蔽字恢复为原来的值



alarm (10) ; 睡了3秒
int unsleep = alarm (20) ;
unsleep = 7; 第一次未睡够的秒数

mask = 阻塞SIGALARM
tmp_mask=解除SIGALARM阻塞
SIGALARM阻塞=20; 设置信号屏蔽字



1.阻塞SIGALRM
alarm(0);
2.
pause();
3.解除阻塞
2和3位置都不能解除阻塞

1.阻塞SIGALRM

sigprocmask(mask); //设置当前阻塞信号集，阻塞SIGALRM

alarm(1);

sigsuspend(tmp_mask); 挂起，然后解除阻塞集，恢复原来阻塞集。

1.挂起自己

2.用临时阻塞信号集替换原有阻塞信号集，解除SIGALRM的阻塞

3.被信号唤醒时恢复当前进程原有的阻塞信号集

return之前

还原SIGALRM的处理动作，还原阻塞信号集

1、mysleep.c(存在BUG)

```
1 #include <stdio.h>
2 #include <signal.h>
3
4 void sig_alarm(int signo)
5 {
6
7 }
8
9 unsigned int mysleep(unsigned int nsecs)
10 {
11     struct sigaction newact,oldact;
12     unsigned int unslept;
13
14     newact.sa_handler = sig_alarm;
15     sigemptyset(&newact.sa_mask);
16     newact.sa_flags = 0;
17     sigaction(SIGALRM,&newact,&oldact);
18
19     //当执行完这条语句之后，CPU的时间用完了，定时器时间到了，触发了捕捉函数，然后得到CPU，挂
20     alarm(nsecs); //定时器一直在运行，即使没有CPU
21     //挂起
22     pause();
23
24     unslept = alarm(0);
25     sigaction(SIGALRM,&oldact,NULL);
26
27     return unslept;
28 }
29
30 int main(void)
31 {
32     printf("hello\n");
33     mysleep(1);
```

```
33     mysleep(1);
34     printf("world\n");
35     return 0;
36 }
```

2、优化的mysleep.c

```
1 #include <stdio.h>
2 #include <signal.h>
3
4 void sig_alarm(int signo)
5 {
6
7 }
8
9 unsigned int mysleep(unsigned int nsecs)
10 {
11     struct sigaction newact,oldact;
12     unsigned int unslept;
13     sigset_t newmask,oldmask,suspmask;
14
15     //捕捉设置
16     newact.sa_handler = sig_alarm;
17     sigemptyset(&newact.sa_mask);
18     newact.sa_flags = 0;
19     sigaction(SIGALRM,&newact,&oldact);
20
21     //设置阻塞集
22     sigemptyset(&newmask);
23     sigaddset(&newmask,SIGALRM);
24     sigprocmask(SIG_BLOCK,&newmask,&oldmask);
25
26     alarm(nsecs);//定时器一直在运行，即使没有CPU
27     //挂起
28     //临时解除阻塞集
29     suspmask = oldmask;
30     sigdelset(&suspmask,SIGALRM);
31     sigsuspend(&suspmask);//挂起，临时解除阻塞集，恢复原来阻塞集
32
33     unslept = alarm(0);
34     sigaction(SIGALRM,&oldact,NULL);
```

```
35
36     return unslept;
37 }
38
39 int main(void)
40 {
41     printf("hello\n");
42     mysleep(1);
43     printf("world\n");
44     return 0;
45 }
```